The CopernicusAI Knowledge Engine MCP Server: Enabling AI-Assisted Research Through Programmatic Knowledge Access

Gary Welz
Retired Faculty Member
John Jay College, CUNY (Department of Mathematics and Computer Science)
Borough of Manhattan Community College, CUNY
CUNY Graduate Center (New Media Lab)
Email: gwelz@jjay.cuny.edu

Abstract

This paper presents the implementation of a Model Context Protocol (MCP) server for the CopernicusAI Knowledge Engine, which exposes all knowledge engine components as queryable tools for AI assistants. The MCP server enables direct programmatic access to research paper metadata, biological process visualizations (GLMP), podcast content, and cross-component relationships through a standardized protocol interface. We describe the architecture, implementation, and 15 available tools that enable AI assistants (Cursor, Claude Desktop, etc.) to query and reason about scientific knowledge across multiple modalities. This implementation demonstrates practical application of "computable artifacts" by making structured knowledge programmatically accessible, supporting both human researchers and AI-assisted workflows.

Keywords: Model Context Protocol, AI-assisted research, knowledge engineering, programmatic access, computable artifacts, scientific knowledge systems

---

1. Introduction

1.1 The Challenge of Knowledge Access

Scientific research generates vast amounts of knowledge across multiple modalities: research papers, experimental protocols, biological pathway diagrams, educational content, and more. While this knowledge is increasingly digitized, accessing and integrating it programmatically remains challenging. Traditional approaches require:

- **Manual browsing** of databases and websites
- **API-specific integrations** for each data source
- **Custom scripts** for each query type
- **Limited cross-modal linking** between related content

The CopernicusAI Knowledge Engine addresses these challenges by integrating multiple knowledge

sources into a unified system. However, to fully realize the potential of this integration, we need programmatic access that enables both human researchers and AI assistants to query and reason about the knowledge base.

1.2 Model Context Protocol (MCP)

The Model Context Protocol (MCP) is a standardized protocol that enables AI assistants to interact with external tools and data sources. MCP servers expose structured tools that AI assistants can discover, invoke, and reason about, enabling seamless integration between AI systems and external knowledge bases.

By implementing an MCP server for the CopernicusAI Knowledge Engine, we enable: - **Direct AI assistant access** to all knowledge engine components - **Unified query interface** across papers, processes, podcasts, and videos - **Real-time cross-component linking** and relationship discovery - **Developer-friendly API** for building custom research tools

1.3 Paper Structure

This paper describes the implementation of the CopernicusAI Knowledge Engine MCP server, including: - Architecture and design decisions - Implementation details for 15 tools across 4 component categories - Integration with existing infrastructure (Firestore, Google Cloud Storage) - Usage examples and applications - Benefits for AI-assisted research workflows

---

2. Background and Motivation

2.1 The CopernicusAI Knowledge Engine

The CopernicusAI Knowledge Engine is an integrated system that combines:

1. **Research Paper Metadata Database:** Centralized repository of scientific literature with AI-powered preprocessing, entity extraction, and citation tracking
2. **GLMP (Genome Logic Modeling Project):** 110+ biological process visualizations stored as structured JSON with Mermaid flowcharts
3. **CopernicusAI Podcasts:** AI-generated scientific podcasts with source paper tracking and RSS distribution
4. **Science Video Database:** (Future) Curated educational videos with transcript-based search
5. **Programming Framework:** (Future) Cross-domain process visualizations

Each component stores structured, metadata-rich data that is ideal for programmatic access.

2.2 The Need for Programmatic Access

While the knowledge engine provides web interfaces for human users, programmatic access enables:

- **AI-assisted research:** AI assistants can query the knowledge base to answer questions, find related content, and verify information
- **Automated workflows:** Scripts and tools can integrate knowledge engine data into research pipelines
- **Cross-component analysis:** Tools can discover relationships across papers, processes, and podcasts
- **Custom applications:** Developers can build specialized interfaces and tools

2.3 Why MCP?

MCP provides several advantages over custom APIs:

- **Standardized protocol:** Works with any MCP-compatible client (Cursor, Claude Desktop, custom tools)
- **Tool discovery:** Clients automatically discover available tools and their schemas
- **Type safety:** Structured input/output schemas ensure correct usage
- **Extensibility:** Easy to add new tools without changing client code
- **AI-native:** Designed specifically for AI assistant interactions

---

3. Architecture and Design

3.1 System Architecture

The MCP server follows a modular architecture with clear separation of concerns:

Top Level: - MCP Client (Cursor, Claude Desktop, etc.) communicates via MCP Protocol over stdio

Server Layer: - MCP Server (server.py) handles tool registration, request routing, and response formatting

Tool Layer: - Papers Tools (4 tools) - Research paper queries - GLMP Tools (4 tools) - Biological process queries
- Podcast Tools (4 tools) - Podcast content queries - Cross-Component Tools (3 tools) - Cross-modal queries

Data Access Layer: - Utility Clients (Firestore Client, GCS Client) provide abstraction over data sources

Storage Layer: - Firestore Database - Paper metadata, podcast metadata - Google Cloud Storage - GLMP process JSON files

3.2 Component Organization

The server is organized into modules:

- **`server.py`:** Main MCP server implementation, tool registration, request routing
- **`config.py`:** Centralized configuration (GCP credentials, collection names, limits)
- **`tools/papers.py`:** Research paper query tools (4 tools)
- **`tools/glmp.py`:** GLMP process tools (4 tools)
- **`tools/podcasts.py`:** Podcast query tools (4 tools)
- **`tools/cross_component.py`:** Cross-component integration tools (3 tools)
- **`utils/firestore_client.py`:** Firestore database utilities
- **`utils/gcs_client.py`:** Google Cloud Storage utilities

3.3 Design Principles

1. **Modularity:** Each component category has its own tool module
2. **Consistency:** All tools follow the same input/output patterns
3. **Error Handling:** Graceful error handling with informative messages
4. **Performance:** Efficient queries with configurable limits
5. **Extensibility:** Easy to add new tools or modify existing ones

---

4. Implementation Details

4.1 Server Setup

The MCP server uses the official MCP Python SDK:

```
from mcp.server import Server
from mcp.server.stdio import stdio_server
from mcp.types import Tool, TextContent

server = Server("copernicusai-knowledge-engine")

@server.list_tools()
async def list_tools() -> list[Tool]:
    # Return list of available tools with schemas
    ...

@server.call_tool()
async def call_tool(name: str, arguments: dict) -> Sequence[TextContent]:
    # Route to appropriate tool implementation
    ...
```

The server communicates via stdio (standard input/output), which is the standard for MCP clients like Cursor and Claude Desktop.

4.2 Tool Registration

Each tool is registered with: - **Name:** Unique identifier (e.g., `query_research_papers`) - **Description:** Human-readable description of what the tool does - **Input Schema:** JSON Schema defining required and optional parameters - **Implementation:** Async function that performs the query and returns results

Example tool registration:

```
Tool(
    name="query_research_papers",
    description="Query research papers by search terms, discipline, or keywords.",
    inputSchema={
        "type": "object",
        "properties": {
            "query": {"type": "string", "description": "Search query"},
            "discipline": {"type": "string", "description": "Filter by discipline"},
            "limit": {"type": "integer", "default": 10, "minimum": 1, "maximum": 100}
        },
        "required": []
    }
)
```

4.3 Data Access Layer

The server uses utility modules to access underlying data sources:

Firestore Client (`utils/firestore_client.py`): - Singleton pattern for connection management - Generic query functions for any collection - Document retrieval by ID - Filtering, ordering, and limiting support

GCS Client (`utils/gcs_client.py`): - List GLMP JSON files in bucket - Retrieve and parse individual process files - Search processes by category, entity, or content

4.4 Response Formatting

All tools return JSON-formatted text responses, making them easy for AI assistants to parse and reason about. Responses include: - Structured data (lists, objects) - Metadata (counts, pagination info) - Error messages (when applicable)

---

5. Available Tools

The MCP server exposes 15 tools across 4 categories:

5.1 Research Paper Tools (4 tools)

1. `query_research_papers` - **Purpose:** Search papers by query, discipline, or keywords - **Parameters:** `query` (string), `discipline` (string), `limit` (integer) - **Returns:** List of matching papers with metadata - **Use Case:** "Find papers about CRISPR in biology"

2. `get_paper_by_id` - **Purpose:** Get full paper metadata by ID or DOI - **Parameters:** `paper_id` (string) or `doi` (string) - **Returns:** Complete paper data including preprocessing, entities, citations - **Use Case:** "Get details for paper with DOI 10.1038/nature12345"

3. `get_paper_citations` - **Purpose:** Find papers that cite a specified paper - **Parameters:** `paper_id` (string) or `doi` (string) - **Returns:** List of citing papers - **Use Case:** "What papers cite the Jacob & Monod 1961 paper?"

4. `search_papers_by_entity` - **Purpose:** Search papers mentioning specific entities (genes, proteins, etc.) - **Parameters:** `entity` (string, required), `entity_type` (string), `limit` (integer) - **Returns:** Papers containing the entity in their extracted entities list - **Use Case:** "Find all papers mentioning p53"

5.2 GLMP Tools (4 tools)

5. `list_glmp_processes` - **Purpose:** List biological processes, optionally filtered by category - **Parameters:** `category` (string), `limit` (integer) - **Returns:** List of processes with metadata (title, category, description) - **Use Case:** "List all DNA repair processes"

6. `get_glmp_process` - **Purpose:** Get full process details including Mermaid diagram - **Parameters:** `process_id` (string) or `process_name` (string) - **Returns:** Complete process data with Mermaid flowchart code - **Use Case:** "Get the beta-galactosidase regulation process"

7. `search_glmp_by_entity` - **Purpose:** Find processes involving specific entities - **Parameters:** `entity` (string, required), `limit` (integer) - **Returns:** Processes containing the entity - **Use Case:** "Find all processes involving ATP"

8. `get_glmp_categories` - **Purpose:** Get all GLMP categories with process counts - **Parameters:** None - **Returns:** List of categories with counts, total processes - **Use Case:** "What categories are available in GLMP?"

5.3 Podcast Tools (4 tools)

9. `list_podcasts` - **Purpose:** List podcast episodes, optionally filtered - **Parameters:** `discipline` (string), `subscriber_id` (string), `limit` (integer) - **Returns:** List of podcast episodes with metadata - **Use Case:** "List all biology podcasts"

10. `get_podcast_details` - **Purpose:** Get full metadata for a specific episode - **Parameters:** `podcast_id` (string, required) - **Returns:** Complete episode data (title, description, audio URL, etc.) - **Use Case:** "Get details for podcast episode abc123"

11. `get_podcast_source_papers` - **Purpose:** Get research papers used as sources for an episode - **Parameters:** `podcast_id` (string, required) - **Returns:** List of source papers with metadata - **Use Case:** "What papers were used for the CRISPR podcast?"

12. `search_podcasts_by_topic` - **Purpose:** Search podcasts by topic or keywords - **Parameters:** `topic` (string, required), `limit` (integer) - **Returns:** Matching podcast episodes - **Use Case:** "Find podcasts about gene editing"

5.4 Cross-Component Tools (3 tools)

13. `find_related_content` - **Purpose:** Find related content across all components - **Parameters:** `paper_id`, `podcast_id`, or `process_id` (one required), `limit` (integer) - **Returns:** Related papers, processes, and podcasts - **Use Case:** "Find content related to this paper"

14. `get_paper_visualizations` - **Purpose:** Get GLMP processes related to a specific paper - **Parameters:** `paper_id` (string) or `doi` (string) - **Returns:** Related GLMP process diagrams - **Use Case:** "What GLMP processes are related to this paper?"

15. `search_across_components` - **Purpose:** Unified search across all components - **Parameters:** `query` (string, required), `components` (array), `limit` (integer) - **Returns:** Results from papers, GLMP, and podcasts - **Use Case:** "Search everything for 'CRISPR' "

---

6. Integration with Existing Infrastructure

6.1 Google Cloud Platform Integration

The MCP server integrates with existing GCP infrastructure:

Firestore Database: - Collection: `research_papers` - Paper metadata - Collection: `podcast_jobs` - Podcast generation jobs - Collection: `episodes` - Podcast episode metadata - Collection: `subscribers` - Subscriber information

Google Cloud Storage: - Bucket: `regal-scholar-453620-r7-podcast-storage` - Path: `glmp-v2/processes/` - GLMP JSON files - Each process stored as individual JSON file with Mermaid code

6.2 Authentication

The server uses Google Cloud Application Default Credentials: - Automatically detects credentials from environment - Works with service accounts for production - Supports local development with

```
gcloud auth application-default login
```

6.3 Configuration

Centralized configuration in `config.py`: - GCP project ID and region - Firestore database name - GCS bucket names and paths - Collection names - Query limits (default: 10, max: 100) - Cache TTL settings

---

7. Usage Examples

7.1 Example 1: Finding Related Content

Scenario: A researcher wants to find all content related to a specific paper about CRISPR.

MCP Tool Calls: 1. `get_paper_by_id(doi="10.1038/nature12345")` - Get paper details 2. `search_papers_by_entity(entity="CRISPR")` - Find related papers 3. `search_glmp_by_entity(entity="CRISP` - Find related processes 4. `search_podcasts_by_topic(topic="CRISPR")` - Find related podcasts 5. `find_related_content(paper_id="...")` - Unified related content

Result: Comprehensive view of all CRISPR-related content across the knowledge engine.

7.2 Example 2: Process Discovery

Scenario: A biologist wants to explore DNA repair processes.

MCP Tool Calls: 1. `get_glmp_categories()` - See available categories 2. `list_glmp_processes(category="DN` `Repair")` - List DNA repair processes 3. `get_glmp_process(process_name="Base Excision Repair")` - Get detailed process 4. `get_paper_visualizations(paper_id="...")` - Find related papers

Result: Detailed DNA repair process with Mermaid flowchart and related papers.

7.3 Example 3: Cross-Modal Research

Scenario: An AI assistant helping with research on a specific topic.

MCP Tool Calls: 1. `search_across_components(query="lactose metabolism")` - Unified search 2. `get_paper_by_id(doi="...")` - Get key paper details 3. `get_glmp_process(process_name="beta-gala` - Get process diagram 4. `get_podcast_source_papers(podcast_id="...")` - Find educational content

Result: Multi-modal view of lactose metabolism: papers, processes, and podcasts.

---

8. Benefits and Applications

8.1 AI-Assisted Research

The MCP server enables AI assistants to: - **Answer questions** by querying the knowledge base - **Find related content** across multiple modalities - **Verify information** by checking multiple sources - **Discover connections** between papers, processes, and podcasts

8.2 Developer Tools

Developers can build: - **Custom research interfaces** using MCP tools - **Automated workflows** that integrate knowledge engine data - **Analysis tools** that query and process knowledge base content - **Third-party integrations** with other research tools

8.3 Educational Applications

Educators can: - **Create interactive lessons** that query real scientific data - **Build study tools** that help students explore topics - **Generate quizzes** from knowledge base content - **Track learning progress** through query patterns

8.4 Research Workflows

Researchers can: - **Automate literature reviews** by querying related papers - **Discover process visualizations** for papers they're reading - **Find educational content** to understand complex topics - **Track citations** and research impact

---

9. Implementation Status

9.1 Completed Phases

Phase 1: Foundation & Setup (Complete) - MCP SDK installation and setup - Server skeleton and configuration - Authentication setup

Phase 2: Research Paper Tools (Complete) - 4 tools implemented and tested - Firestore integration working - Entity extraction search functional

Phase 3: GLMP Tools (Complete) - 4 tools implemented and tested - GCS integration working - Category and entity search functional

Phase 4: Podcast Tools (Complete) - 4 tools implemented and tested - Firestore integration working - Source paper linking functional

Phase 5: Cross-Component Tools (Complete) - 3 tools implemented and tested - Cross-component linking working - Unified search functional

9.2 Current Status

- **15 tools operational** and ready for use
- **Deployed locally** for Cursor IDE integration
- **Documentation complete** (README, User Guide, Deployment Guide)
- **Testing framework** in place

9.3 Future Enhancements

Phase 6: Testing & Validation (Planned) - Comprehensive unit tests - Integration tests - Performance testing - Load testing

Phase 7: Production Deployment (Planned) - Cloud Run deployment - Monitoring and logging - Rate limiting - Authentication/authorization

Future Tools: - Science Video Database tools - Programming Framework tools - Advanced analytics tools - Real-time update notifications

---

10. Technical Considerations

10.1 Performance

- **Query limits:** Default 10, maximum 100 results per query
- **Caching:** Configurable TTL for frequently accessed data
- **Efficient queries:** Indexed Firestore queries, optimized GCS listing
- **Async operations:** All I/O operations are asynchronous

10.2 Error Handling

- **Graceful degradation:** Tools return error messages instead of crashing
- **Informative errors:** Error messages include context and suggestions
- **Logging:** All errors logged for debugging and monitoring

10.3 Security

- **Authentication:** Uses GCP Application Default Credentials
- **Authorization:** (Future) Role-based access control
- **Input validation:** All inputs validated against schemas
- **Rate limiting:** (Future) Prevent abuse

10.4 Extensibility

- **Modular design:** Easy to add new tools or modify existing ones
- **Standardized patterns:** Consistent structure across all tools

- **Configuration-driven:** Behavior controlled via config file
- **Plugin architecture:** (Future) Support for custom tool plugins

---

11. Comparison with Alternative Approaches

11.1 Custom REST API

MCP Advantages: - Standardized protocol (works with any MCP client) - Tool discovery (clients auto-discover capabilities) - AI-native design (optimized for AI assistant interactions) - Type safety (structured schemas)

REST API Advantages: - More familiar to web developers - Better for web applications - More flexible for custom use cases

Our Choice: MCP for AI assistant integration, with potential REST API wrapper for web applications.

11.2 Direct Database Access

MCP Advantages: - Abstraction layer (hides implementation details) - Security (controlled access, no direct DB access) - Consistency (standardized query patterns) - Extensibility (easy to add logic, caching, etc.)

Direct DB Advantages: - Lower latency (no server layer) - More control (direct query optimization) - Simpler architecture (fewer components)

Our Choice: MCP for controlled, secure access with abstraction benefits.

---

12. Conclusion

The CopernicusAI Knowledge Engine MCP server demonstrates practical implementation of programmatic knowledge access for AI-assisted research. By exposing 15 tools across 4 component categories, the server enables:

- **AI assistants** to query and reason about scientific knowledge
- **Developers** to build custom research tools and integrations
- **Researchers** to automate workflows and discover connections
- **Educators** to create interactive learning experiences

The implementation follows best practices for modularity, extensibility, and performance, while maintaining compatibility with the MCP standard for broad client support.

Future Work: - Production deployment with monitoring and scaling - Additional tools for video database and programming framework - Advanced features (real-time updates, analytics, etc.) - Community contributions and extensions

The MCP server represents a significant step toward making scientific knowledge truly "computable"—accessible not just to humans through web interfaces, but to AI systems and automated tools through programmatic APIs.

Acknowledgments

The MCP server builds upon the CopernicusAI Knowledge Engine infrastructure and integrates with Google Cloud Platform services. We thank the MCP community for the standardized protocol and development tools.

References

1. Model Context Protocol (MCP). (2024). https://modelcontextprotocol.io/
2. CopernicusAI Knowledge Engine. (2024). https://www.copernicusai.fyi
3. Google Cloud Firestore. (2024). https://cloud.google.com/firestore
4. Google Cloud Storage. (2024). https://cloud.google.com/storage
5. Welz, G. (2024). The Programming Framework: A General Method for Process Analysis Using LLMs and Mermaid Visualization. *Hugging Face Space*. https://huggingface.co/spaces/garywelz/programming
6. Genome Logic Modeling Project (GLMP). (2024). *Hugging Face Space*. https://huggingface.co/spaces/garywelz
7. MCP Python SDK. (2024). https://github.com/modelcontextprotocol/python-sdk
8. Cursor IDE. (2024). https://cursor.sh/
9. Claude Desktop. (2024). https://claude.ai/desktop

Appendix A: Tool Schema Examples

Example 1: query_research_papers

```
{
  "name": "query_research_papers",
  "description": "Query research papers by search terms, discipline, or keywords.",
  "inputSchema": {
```

```
    "type": "object",
    "properties": {
      "query": {
        "type": "string",
        "description": "Search query string (searches in title, abstract, keywords)"
      },
      "discipline": {
        "type": "string",
        "description": "Filter by discipline (e.g., 'biology', 'chemistry', 'physics')"
      },
      "limit": {
        "type": "integer",
        "description": "Maximum number of results (default: 10, max: 100)",
        "default": 10,
        "minimum": 1,
        "maximum": 100
      }
    },
    "required": []
  }
}
```

Example 2: get_glmp_process

```
{
  "name": "get_glmp_process",
  "description": "Get full GLMP process details including Mermaid diagram.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "process_id": {
        "type": "string",
        "description": "GLMP process ID"
      },
      "process_name": {
        "type": "string",
        "description": "Process name (alternative to process_id)"
      }
    },
    "required": []
```

```
  }
}
```

Appendix B: Response Format Examples

Example 1: query_research_papers Response

```
{
  "papers": [
    {
      "id": "abc123",
      "title": "CRISPR-Cas9 Gene Editing",
      "authors": ["Smith, J.", "Jones, M."],
      "doi": "10.1038/nature12345",
      "discipline": "biology",
      "abstract": "We describe a novel CRISPR-Cas9 system...",
      "keywords": ["CRISPR", "gene editing", "Cas9"]
    }
  ],
  "count": 1,
  "query": "CRISPR"
}
```

Example 2: get_glmp_process Response

```
{
  "process": {
    "id": "beta-galactosidase-regulation",
    "title": "Beta-Galactosidase Regulation System",
    "description": "Regulatory system controlling lactose metabolism...",
    "category": "Gene Regulation",
    "mermaid": "flowchart TD\n    A[Lactose Present] --> B[LacI Repressor]\n    ...",
    "entities": ["LacI", "beta-galactosidase", "lactose"],
    "version": "1.0"
  }
}
```

Appendix C: Installation and Setup

Prerequisites

- Python 3.10+
- Google Cloud SDK (for authentication)

- Access to CopernicusAI GCP project

Installation Steps

1. **Install dependencies:**

   ```
   cd cloud-run-backend
   pip install -r mcp_server/requirements.txt
   ```

2. **Configure authentication:**

   ```
   gcloud auth application-default login
   ```

3. **Set environment variables (optional):**

   ```
   export GCP_PROJECT_ID="regal-scholar-453620-r7"
   export FIRESTORE_DATABASE="copernicusai"
   export GCP_AUDIO_BUCKET="regal-scholar-453620-r7-podcast-storage"
   ```

4. **Test server:**

   ```
   python -m mcp_server.server
   ```

Cursor IDE Integration

1. Create `~/.config/cursor/mcp.json`:

   ```
   {
     "mcpServers": {
       "copernicusai": {
         "command": "python3",
         "args": ["-m", "mcp_server.server"],
         "cwd": "/path/to/copernicus-web-public/cloud-run-backend",
         "env": {
           "GCP_PROJECT_ID": "regal-scholar-453620-r7",
           "FIRESTORE_DATABASE": "copernicusai",
           "GCP_AUDIO_BUCKET": "regal-scholar-453620-r7-podcast-storage"
         }
       }
     }
   }
   ```

2. Restart Cursor IDE

3. Verify connection by asking: “What MCP tools are available from CopernicusAI?”